PaperDatabase: a text-based, verifiable knowledge base for assisted reading of scientific papers

Léonard Rivals Maksym Lytvynenko

26 September 2026

Abstract

Preparing a PhD means reading, comparing and citing dozens of papers. A large language model (LLM) can help, but it works poorly on raw PDFs: it rereads the full text for every question, does not know where to look for a number, and mixes what a paper says with what it infers. PaperDatabase converts each PDF into structured text files (metadata, paginated sections, references, reading note, benchmark table) that an LLM agent, here Claude Code, walks from summary to detail. A passage-retrieval layer, in the RAG style, sits on top: a BM25 lexical index and a dense index (bge-small-en-v1.5) merged by Reciprocal Rank Fusion, with optional reranking. This layer generates nothing. It returns identified, paginated and replayable passages, plus a command that checks that a quotation, and in particular its numbers, does appear in the cited passage. Writing stays with the agent, bound by sourcing rules. A reproducible evaluation bench, with no LLM and no network, measures extraction, retrieval, verification and timings. On a corpus of 45 papers (3 479 passages), hybrid retrieval with reranking reaches a recall@5 of 0.80 on a held-out question set that was never used for tuning, versus 1.00 on the set that guided the design, and 0.50 in French. Growing from 28 to 45 papers, with the same questions, lowers hybrid retrieval without reranking from 0.80 to 0.72 on the held-out set. A second, optional GPU engine (Qwen3-Embedding-4B and Qwen3-Reranker-4B) retrieves all 8 French questions, versus 5 for the best CPU mode. Verification now rejects every quotation with an altered number, which it used to accept. No simple signal proved reliable for detecting that a question falls outside the corpus. A real sourced-answer trial consumed about 2 500 tokens, whereas a single full text is about 13 000 tokens (median).

Contents

1 Introduction **3**

2 Design principles **3**

3 Architecture and pipeline **4**
3.1 Acquisition and ingestion 4
3.2 Data model 5
3.3 Modules 5

4 The retrieval layer (RAG) **6**
4.1 A RAG without generation 6
4.2 Passage chunking 7
4.3 Two indexes and a fusion 7
4.4 Commands 8
4.5 Answer protocol: the `/paper-ask` skill 9
4.6 A second GPU engine and open-access acquisition 9

5 What the software brings **10**

6 Tests and evaluation **11**
6.1 Unit tests 11
6.2 The `pdb eval` bench 12
6.3 Extraction quality 12
6.4 Passage retrieval 13
6.5 Detecting that a question falls outside the corpus 15
6.6 Quotation checking 16
6.7 Timings 16
6.8 Real end-to-end trial 17
6.9 Synthetic benchmark (`pdb synth`) 17
6.10 Threats to validity 17

7 Limitations and outlook **18**

8 Conclusion **19**

A Acronyms **20**

B Glossary **21**

C Commands **22**

1 Introduction

Context. This work stems from a concrete need: preparing a PhD proposal on embedded vision on FPGAs (YOLO detectors, autoencoders, continual learning). The field publishes many numbers (frames per second, GOPS, watts, LUT, DSP and BRAM resources), measured under heterogeneous conditions. Comparing them requires knowing where each value comes from. The corpus of this use case serves as a test bed, but the tool does not depend on the domain: axes, tags and benchmark columns are redefined in a few constants.

Problem. An LLM agent given PDFs has three flaws:

1. *cost*: it rereads full texts (about 13 000 tokens each, median) to answer a question that fits in three sentences;
2. *traceability*: a number or a quotation does not carry the exact place it comes from, and a paraphrased quotation passes for a verbatim one;
3. *confusion between read and inferred*: the agent fills gaps with its general knowledge without saying so.

Approach. PaperDatabase does not try to replace the agent. It gives it a base in which every piece of information carries its source, and deterministic commands that answer factual questions without calling an LLM. Generation is limited to questions that need a synthesis, and it works from outputs that are already sourced.

Contributions.

- An idempotent PDF → structured text pipeline (GROBID, PyMuPDF fallback, ligature repair, table reconstruction) that never guesses missing metadata (section 3).
- A fully textual data model, validated at every reindex, where every numerical value carries its source section and a confidence level (section 3.2).
- A hybrid retrieval layer without generation, organised in three cost levels and equipped with automatic quotation checking (section 4).
- A reproducible evaluation bench, with no LLM and no network, that measures the quality of extraction, retrieval and quotation checking, as well as timings, with a held-out question set never used to tune any parameter; and a unit-test suite that runs without models (section 6).
- A second, optional GPU retrieval engine (Qwen3) and user-approved open-access paper acquisition (section 4.6), compared with the first engine on the same question sets.

2 Design principles

The final consumer of the base is not a search engine. It is an LLM agent that already has file-reading and text-search tools (`grep`, `read`). On a corpus of a few dozen to a few hundred papers, a well-named directory tree, an index and a filterable CSV are enough for most questions, and they remain readable, diffable from one version to the next and repairable by hand. Four rules follow.

1. **Everything is text.** No database and no binary format, apart from the original PDF, kept as evidence. The retrieval index (section 4) is a derived, rebuildable cache, outside the versioned tree.

2. **Every number is traceable.** Every value in the benchmark table carries the section, table or figure it comes from. A value not reported by the authors is written `n/r`, never an estimate.

3. **Read from summary to detail.** The agent reads the highest layer that answers and only goes down when needed (table 1). It never reopens the PDF.

4. **The schema is coded in a single place** (`lib/schema.py`) and checked at every reindex, which stops at the first discrepancy.

Table 1: Reading layers, from summary to detail.

Layer	File or command	Used to
1	`INDEX.md` (generated)	know which papers exist, by axis, with or without a note
2	`benchmarks.csv` via `pdb bench`	find a number, with its source section
3	`papers/<id>/fiche.md`	read the critical analysis of a paper
4	`pdb search`, `show`, `verify`	find and quote the 2 or 3 passages that answer
–	`fulltext.md`	full text, as a last resort

An instruction file (`CLAUDE.md`) describes this reading order to the agent, together with working rules: never invent a reference, write `n/r`, separate what is read from what is inferred, flag preprints and non-comparable metrics.

3 Architecture and pipeline

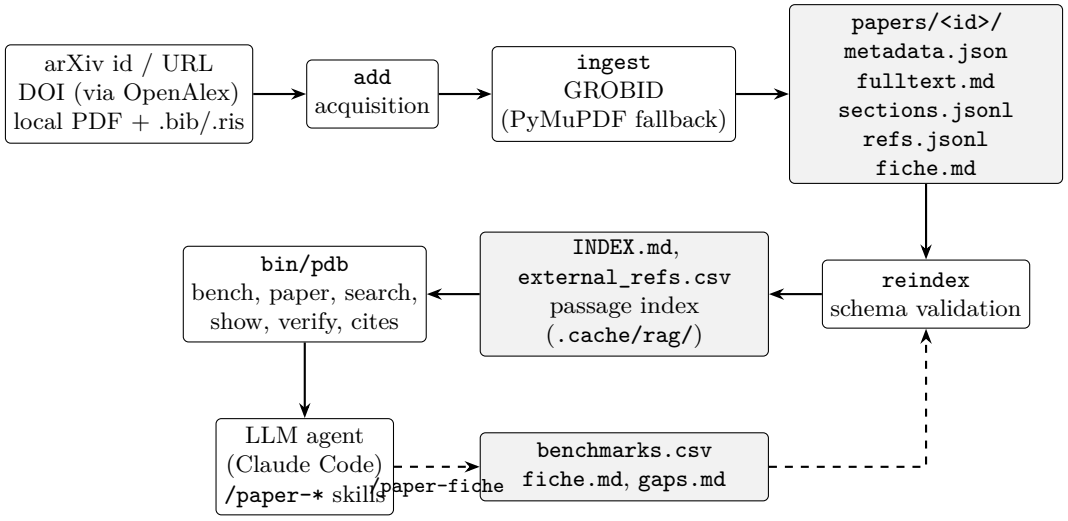


Figure 1: Data flow. Solid arrows are automatic; dashed arrows go through the agent, which writes reading notes and benchmark rows from the extracted text.

3.1 Acquisition and ingestion

`bin/add.py` fetches an arXiv paper (metadata through the API, then the PDF), resolves a DOI to its arXiv version through OpenAlex (Priem et al., 2022), or copies a local PDF. Paywalled papers are dropped in by hand: the tool bypasses no access control.

`bin/ingest.py` sends each PDF to GROBID (GROBID contributors, 2008–2026), which returns a TEI structure (header, sections, figures, bibliography). Without GROBID, a PyMuPDF

fallback splits the text heuristically on numbered headings and records this in the metadata. The pipeline then applies, in order:

- *metadata merging* by order of trust: publisher citation export, then arXiv, then GROBID. An unknown field stays `null`;
- *cleaning*: repeated headers and footers, page numbers, bibliography moved out of the body into `refs.jsonl`;
- *repair of lost ligatures* (« classi cation » for « classification »): a word is rejoined only if the result is known to the corpus or to a built-in list and the fragment is not;
- *tables*: a table for which GROBID kept only the caption goes through two successive fallbacks. PyMuPDF first looks for it on its page (ruled tables). Docling (Auer et al., 2024) comes next, on request (`ingest --reparse --tables docling`), with optional optical character recognition (OCR) for tables published as images. A reconstructed body is labelled "reconstructed by PyMuPDF / Docling: to be checked", and a table left empty is reported by `pdb status`;
- *misclassified paragraphs*: a paragraph that GROBID took for a figure is returned to the body when it looks like prose.

Ingestion is idempotent, and a re-extraction (`--force`, or `--reparse` from the stored TEI) never overwrites a reading note already written nor a classification corrected by hand.

3.2 Data model

Each paper becomes a folder `papers/<paper_id>/`. The identifier is stable and readable (`arxiv-<id>-<slug>` or `<year>-<author>-<slug>`). The extracted text exists in two forms: `fulltext.md`, human-readable, and `sections.jsonl`, one line per section with its type (abstract, section, figure, table) and its page. The latter is the raw material of the passage index.

The file `benchmarks.csv` has one row per reported configuration: a paper that presents three variants yields three rows. Besides the measurement columns (model, platform, quantisation, FPS, GOPS, latency, power, LUT, DSP, BRAM, dataset, toolflow), three columns are used to judge each value:

- `source_section`: section, table or figure of origin;
- `confidence`: `high` (read from a table), `medium` (read from the text) or `low` (inferred, to be rechecked);
- `power_scope`: scope of the reported power (`chip`, `board`, `board-dynamic`, `estimated`, `unspecified`). Two powers with different scopes cannot be compared: a board can draw several watts at idle, which a "board" measurement includes and a "chip" measurement excludes.

Reading notes (`fiche.md`) follow a fixed template (problem, method, results, limitations, relevance) with a mandatory "reading status" section that separates what was read from what was inferred. They are written by the agent (`/paper-fiche` command) and reviewed by the user.

3.3 Modules

All commands go through a single entry point, `bin/pdb`. Their output follows the same conventions: TSV by default, a header line without banner, notes prefixed by `#`, short paper identifiers (`1612.07119`, `2022-feng`) with their legend at the bottom, and an exit code that distinguishes "results", "no result" and "argument error".

Table 2: Main modules of `lib/`. Everything is written in Python; only dense retrieval, the GPU engine, the figures and the visualisation have optional dependencies.

Module	Role
`tei.py`, `doc.py`, `pdftext.py`	reading GROBID output, common representation, PyMuPDF fallback
`ligatures.py`, `tables.py`, `docling_tables.py`	ligature repair, table reconstruction (PyMuPDF, then Docling)
`citation.py`, `openalex.py`	`.bib`/`.ris` exports, OpenAlex client with cache
`schema.py`	data-model validators, the only place where it is coded
`refs.py`	reference matching (arXiv, DOI, normalised title), internal citations
`query.py`	structured queries without LLM (`bench`, `paper`, `cites`)
`rag.py`	passages, BM25, dense index, fusion, reranking, quotation checking
`rag_qwen.py`, `rag_models.py`	second, GPU engine: FTS5, Qwen3-Embedding-4B, Qwen3-Reranker-4B
`rag_external.py`	staged open-access acquisition (OpenAlex, arXiv), import after approval
`evaluation.py`	evaluation bench (`pdb eval`): extraction, retrieval, `verify`, timings
`status.py`	state of the base and next actions
`plotdata.py`, `plotfig.py`	static figures (`pdb plot`): benchmarks, evaluation, corpus
`graph.py`, `lineage.py`, `vizfig.py`, `embed.py`	3D visualisation and citation lineage (secondary tool)

4 The retrieval layer (RAG)

4.1 A RAG without generation

In a *Retrieval-Augmented Generation* system (Lewis et al., 2020), the LLM receives, together with the question, the passages of a corpus that answer it, and writes from them. PaperDatabase separates the two halves. The `bin/pdb` commands do the retrieval, deterministically and without calling an LLM. The agent calls them, reads their output and writes only when a synthesis is needed. This choice adds neither an API key nor a cost: the generation is done by the agent already in use.

Three goals guide the design, in order of priority: *verifiability* (every returned item carries a replayable source), *token economy* (answering without reading full texts), and *access by meaning* (finding "continual learning" when searching for "incremental learning", which `grep` cannot do).

Table 3: Three query levels, from cheapest to most expensive. One moves up a level only if the previous one is not enough. Costs measured on the current corpus.

Level	Source	LLM	Typical cost
N0 – structured query	`benchmarks.csv`, metadata	no	100–500 tokens
N1 – passage retrieval	passage index	no	400–800 tokens, +300 per opened passage
N2 – synthesis	N0 and N1 outputs only	yes	that of the writing

An N0 or N1 output is often already an answer: a filtered table or three sourced passages are enough for a factual question, and the agent can pass them on without rephrasing. Level N2 adds no information: it links what N0 and N1 returned, and labels its inferences as such.

4.2 Passage chunking

The starting unit is an entry of `sections.jsonl`, which already follows paragraphs. An entry longer than 250 words is split into 200-word windows with a 40-word overlap, because embedding models truncate beyond 256 to 512 tokens. Each passage gets a replayable identifier, `<section>.<k>` (for example `1612.07119#019.3`), and keeps its paper, section, page and the SHA-1 hash of its text. The indexed text is prefixed with the section title, which adds context without changing the returned text. The current corpus has 3 479 passages.

4.3 Two indexes and a fusion

Lexical index. A hand-written BM25 (Robertson and Zaragoza, 2009) ($k_1 = 1.5$, $b = 0.75$, lower-case alphanumeric tokens) finds exact terms: board names (`ZCU104`), model names (`YOLOv5n`), acronyms and numbers.

Dense index. The `BAAI/bge-small-en-v1.5` model (Xiao et al., 2024) (384 dimensions), loaded with sentence-transformers (Reimers and Gurevych, 2019), encodes each passage into a normalised vector. It finds paraphrases and, to some extent, queries in another language. The query gets the instruction prefix recommended for the model; passages do not.

Fusion. Both rankings are merged by Reciprocal Rank Fusion (Cormack et al., 2009):

$$\text{score}(d) = \sum_{i \in \{\text{BM25, dense}\}} \frac{1}{60 + \text{rank}_i(d)}. \tag{1}$$

This method does not depend on score scales, needs no tuning and stays deterministic. Ties are broken by the smallest identifier: the same query on the same index always gives the same ranking.

Absolute relevance. The RRF score is a rank, not a relevance measure: `search` always returns k passages, even when the base does not cover the topic. The output shows the cosine between the query and each passage. A first version warned (`# pertinence faible`, "low relevance") when the best cosine fell below a threshold calibrated by hand on a few queries. The evaluation bench showed that this warning never fired, and that no threshold, neither on the cosine nor on a cross-encoder score, separated out-of-corpus questions from the others (section 6.5). It was therefore removed, because it gave false reassurance: the agent judges, from the snippets, whether the base answers, and says so otherwise instead of filling the gap.

Reranking (option). `--rerank` reorders the top 30 results with the `ms-marco-MiniLM-L-6-v2` cross-encoder, a MiniLM (Wang et al., 2020) trained on MS MARCO (Nguyen et al., 2016) following Nogueira and Cho (2019). It costs about 1.4 s per warm search. It is not enabled by default, for the reasons given in section 6.4.

Storage. Vectors are stored in `.cache/rag/<model>.npz`, keyed by `chunk_id:sha1`. Updates are incremental: only new or modified passages are encoded (about 3 min 40 s for a full build on CPU, i.e. 16 passages per second; 0.2 s with no change). Passages and BM25 are recomputed by each command in under a second. Without sentence-transformers, or if the index is stale, `search` falls back to lexical only and says so. At this scale, an exhaustive numpy search takes a few milliseconds: a dedicated vector database would bring nothing before roughly 10^5 passages (estimate, not measured).

4.4 Commands

`pdb bench` filters `benchmarks.csv` (substrings, numerical bounds, non-empty columns, sorting, aggregation with `--count`). The source columns (`source_section`, `confidence`, `power_scope`) are always shown, even when not requested, and a low-confidence row is prefixed with `!`.

```
$ bin/pdb bench --platform kv260 --cols model,quantization,fps,power_w
paper       model                 quantization  fps  power_w  power_scope  source_section  confidence
2507.18174  YOLOv5 (variante n/r)  INT8          9    3.5      unspecified  Table I         high
# ids: 2507.18174=arxiv-2507.18174-real-time-object-detection-classification
```

`pdb search` returns one line per passage: identifier, section, page, cosine and a snippet of about 160 characters centred on the rarest query term (rare terms in guillemets). Real output, shortened snippets:

```
$ bin/pdb search "power measured at the wall versus the FPGA rails" -k 3
chunk               section                        page  cos   snippet
1612.07119#022.0    5. EVALUATION / 5.2.2. ...      8     0.78  ...both the «wall» «power» and «FPGA» «power
    ↪ » readings...
2025-hozhabr#051.1  V. RESULTS ANALYSIS ... / B. ...  31    0.77  ...computational «power», or throughput, «
    ↪ measured» in...
1612.07119#019.3    5. EVALUATION / 5.1. ...        7     0.73  ...PMBus interface to monitor the «FPGA» «
    ↪ power» supply «rails»...
```

`pdb show` prints the exact text of a passage with its header (section, page, start of the SHA-1) and, on request, its neighbours. `pdb verify` checks that a quotation appears in the cited passage. Both texts are normalised (ligatures, case, hyphenation, everything that is neither letter nor digit), then compared by exact search or, failing that, by the best `difflib` window. The verdict is one of five (the tool's labels are in French):

- `OK`: the quotation appears as is, after normalisation;
- `APPROX`: similarity of at least 0.85 (close paraphrase);
- `NOMBRES` ("numbers"): similarity of at least 0.85, but the numbers of the quotation are not those of the passage;
- `ABSENT`: similarity below 0.85;
- `MODIFIÉ` ("modified"): the passage has changed since its SHA-1 was recorded.

Only `OK` and `APPROX` give a zero exit code. The `NOMBRES` status comes from the evaluation bench. In a long sentence, a changed number keeps the similarity above the threshold, and the first version thus accepted every quotation with an altered number (section 6.6). The added check requires the numbers of the quotation to form a contiguous run of the passage's numbers. Each must appear with its neighbouring words: "Section 3 reviews" is rejected even if the passage contains a 3 elsewhere. Since normalisation removes punctuation, this check is also what tells "94.9 %" from "949 %".

A quotation absent from the cited passage but present elsewhere in the same paper is reported with the identifier where it was found, and the command exits with an error: the source was wrong.

```
$ bin/pdb verify 1612.07119#019.3 "P-wall using a wall-power meter for the whole board power consumption"
APPROX  0.95  1612.07119#019.3
```

Finally, `pdb paper` gives the identity of a paper (authors, venue, preprint or not, note status), and `pdb cites` / `cited-by` walk the citation graph internal to the corpus, rebuilt by matching references on arXiv identifier, DOI or normalised title.

4.5 Answer protocol: the `/paper-ask` skill

The `/paper-ask` skill gives the agent a fixed procedure and only allows it to use the `bin/pdb` commands and file reading:

1. numerical question: `pdb bench` first (N0);
2. otherwise, or in addition: `pdb search --rerank` with one or two English phrasings; if no snippet answers, say that the base probably does not cover the topic instead of filling the gap;
3. `pdb show` on 2 or 3 passages at most, never on the full text;
4. answer with `[chunk_id]` citations, and `pdb verify` on every verbatim quotation: an `ABSENT` quotation is removed, a `NOMBRES` quotation is corrected from the exact text or removed, a wrong identifier is corrected, and an `APPROX` quotation is kept only if presented as a paraphrase;
5. imposed structure: **Read** (sourced), **Inferred**, **Not found in the base**, flagging low-confidence values, preprints and heterogeneous power scopes.

4.6 A second GPU engine and open-access acquisition

Engine. `bin/pdb rag` offers a second retrieval chain, independent from the first and meant for a machine with a GPU (measurements made on a 32 GB RTX 5090): an SQLite FTS5 lexical index, a `Qwen3-Embedding-4B` dense index, the same RRF fusion, then reranking by `Qwen3-Reranker-4B`, in BF16 with pinned model revisions. The index, also stored in `.cache/rag/`, holds *typed* evidence: abstracts, sections, tables and figures of the original text, but also finished reading notes and `benchmarks.csv` rows, each labelled as transcription or note. Results reflect this: a note or a benchmark row is for navigation, and the agent must go back to the original passage (`rag evidence ID --parent`) before quoting. `rag benchmarks` filters numbers with no cap on results and keeps `n/r`, units and power scope; `rag check-answer` checks, for an answer recorded in JSON, that each quoted excerpt appears in the cited evidence. The `/paper-research` skill frames the use of this engine as `/paper-ask` does for the first one.

Acquisition. When the corpus is not enough, `rag acquire` looks for open-access full texts (OpenAlex, arXiv), downloads them into a private staging directory, extracts them and deduplicates them by DOI, arXiv identifier and document hash. Nothing enters the base at this stage: the agent presents each retained paper (title, year, excerpt) and calls `rag approve` only for those the user accepts. An imported paper gets a `todo` note and never overwrites a paper already present. A budget bounds each session: 8 searches, 8 PDFs, 50 MB per PDF, 200 MB and 15 minutes in total. An unavailable PDF is treated as a limitation of the search, not as evidence of absence.

Measurement. The two engines were compared on the question sets of section 6.4, with the same expectations and the same metrics (table 4). This comparison dates from 26/09/2026, on the 28-paper corpus: it was not redone on 45 papers, as the machine used for the second measurement has no GPU.

The only clear gap is French: Qwen retrieves all 8 questions without rephrasing them in English. On the held-out set, it gains 2 questions out of 25 at recall@5, with a comparable MRR. On the dev set, it does worse, which is partly expected since this set was used to tune the first engine. Its lexical channel (FTS5) is weaker than the first engine's BM25 (dev 0.80 versus 0.95; French 0.00). The Qwen reranker is also the best out-of-corpus signal measured (section 6.5). It comes at a cost: a 32 GB GPU, and models reloaded for each query, i.e. 3 to 4 s per question with reranking. The first engine therefore remains the default; the second is an aid for French questions and vocabulary mismatch.

Table 4: CPU engine (bge-small) versus GPU engine (Qwen3), same question sets, 28-paper corpus. Bold: best value per set.

Set	Engine	recall@5	recall@10	MRR	nDCG@10	s / question
dev (20)	bge-small hybrid + reranking	**1.00**	**1.00**	**0.85**	**0.46**	∼1.4
	Qwen hybrid	0.90	0.90	0.78	0.37	0.7
	Qwen hybrid + reranking	0.80	0.80	0.76	0.35	3.7
holdout (25)	bge-small hybrid + reranking	0.80	0.80	**0.78**	**0.41**	∼1.4
	Qwen hybrid + reranking	**0.88**	**0.92**	0.75	0.35	3.3
FR (8)	bge-small dense (best mode)	0.62	0.75	0.46	0.19	0.1
	Qwen dense	**1.00**	**1.00**	0.63	0.22	0.7
	Qwen hybrid + reranking	**1.00**	**1.00**	**0.88**	**0.31**	3.8

5 What the software brings

Token savings. Table 5 compares, for four common tasks, reading the files directly and going through the commands. Orders of magnitude are measured on the current corpus (file size divided by four).

Table 5: Token cost of common tasks (orders of magnitude).

Task	Without the retrieval layer	With
A number on a platform	read the whole `benchmarks.csv`: ∼15 000	filtered `bench`: 100–400
Find where a topic is discussed	`grep` over sections, which misses synonyms	`search -k 8`: ∼600
Read the evidence for a claim	open `fulltext.md`: ∼13 000 (median)	`show` on 1 to 3 passages: 300–900
Check a quotation	reread the passage	`verify`: ∼20

Verifiability. Every returned item can be replayed: a passage by its identifier (`show`) and its page in the PDF, a quotation by `verify`, a number by its `bench` row and source section, a ranking by the determinism of the fusion. An empty result is presented as a statement about the base, not about the literature. Paraphrases, cross-paper connections and inferences cannot be checked automatically: the agent must label them as inferred.

Beyond retrieval. The same base serves other PhD-preparation tasks:

- extracted references feed `external_refs.csv`, which ranks works missing from the base by the number of corpus papers that cite them: these are the next readings;
- `pdb discover` (`/paper-discover`) queries OpenAlex (incoming citations, search by axis or by gap) and proposes candidates, added only with the user's approval;
- `/paper-gaps` keeps a map of uncovered areas (`gaps.md`), each entry supported by paper identifiers;
- a local visualisation app (3D corpus graph, citation timeline) gives the user an overview; the agent does not use it;
- `pdb plot` produces static figures (PNG, PDF or SVG) from `benchmarks.csv`, the latest `pdb eval` report and the corpus metadata: throughput versus power, GOPS/W by precision, FPGA resources, accuracy versus throughput, retrieval quality, years, axes, citations, gaps.

Each figure states its source and sample size in a footer (for example 52 rows out of 172, `n/r` values being excluded), and `figures/manifest.json` lists them. Figure 2 is an example; heterogeneous power scopes are distinguished there rather than mixed.

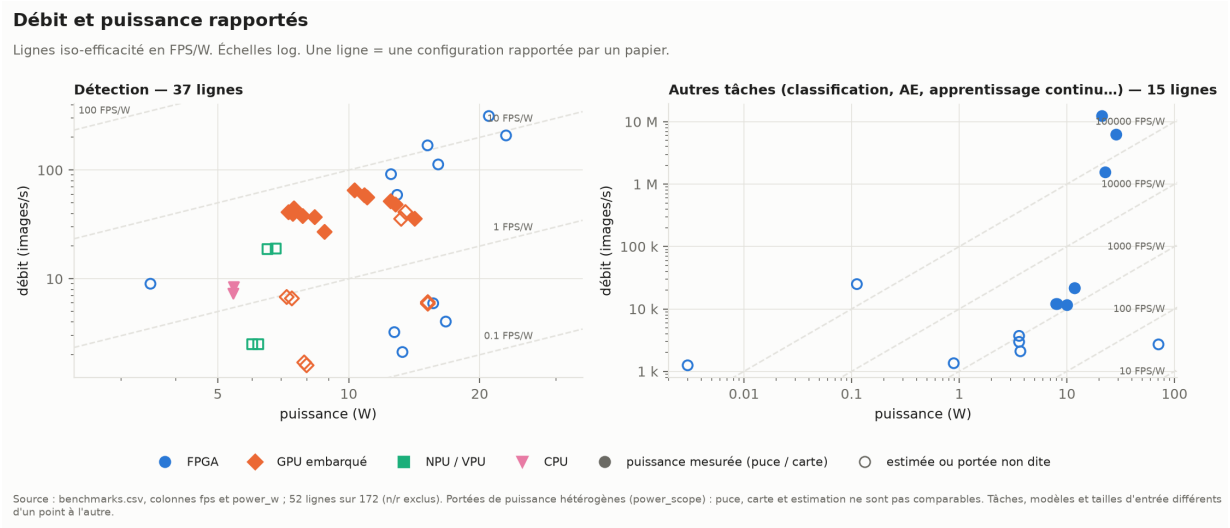


Figure 2: Output of `pdb plot bench` (labels in French): reported throughput and power, one configuration per point (52 rows of `benchmarks.csv` out of 172). Filled markers: measured power; hollow: estimated or unspecified scope. Points cannot be compared with one another without reading the scope of the measurement.

State of the corpus. As of 26 September 2026: 45 papers, 41 of them with a finished reading note, 172 benchmark rows, 3 479 indexed passages, and 25 recorded gaps, 10 of them supported by corpus papers (output of `pdb status`). The corpus has grown by 17 papers since the first version of this text (28 papers, 2 414 passages); the notes and benchmark rows of the new papers were written by the agent.

6 Tests and evaluation

6.1 Unit tests

The suite (`bin/pdb test`) has 183 tests in 20 files. It runs in about twenty seconds, without GROBID and without embedding models, thanks to deterministic stand-ins (table 6):

- a real GROBID (0.8.1) TEI output, kept as a fixture, for parsing;
- a fake encoder and cross-encoder, deterministic functions of the text, to test fusion, reranking and the incremental index without sentence-transformers;
- simulated OpenAlex responses for discovery and addition by DOI;
- a synthetic PDF produced by PyMuPDF (a caption followed by a ruled table) for table reconstruction, and hand-built grids for matching Docling tables;
- small synthetic corpora (including a dated citation chain) for the graph and the lineage.

Visualisation and figure tests are skipped if their optional dependencies are not installed. One test breaks the isolation: the open-access acquisition test (`test_rag_qwen.py`) loads the Qwen tokenizer to chunk the imported paper, and fails on a machine without network where this

Table 6: Unit-test coverage (number of test functions per file).

File	Tests	What is checked
`test_query.py`	16	`bench` filters, sorting and aggregation, forced source columns, short ids, `paper`
`test_tables.py`	14	table reconstruction (synthetic PDF), Docling table filters and matching
`test_schema.py`	13	data-model validators, heading hierarchy
`test_tei.py`	12	parsing of a real GROBID output
`test_openalex.py`	12	discovery, addition by DOI and by list (simulated responses)
`test_citation.py`	11	`.bib`/`.ris` exports: reading, matching, priority
`test_refs.py`	11	reference matching, `external_refs.csv`, internal citations
`test_rag.py`	12	chunking, BM25, fusion, reranking, incremental index, `verify` and number check
`test_evaluation.py`	12	bench metrics on small built cases: coverage, table values, ranks, thresholds, `verify` cases
`test_plot.py`	10	platform families, precision classes, `n/r` exclusion, rendering of every figure on the real base
`test_lineage.py`	9	lineage on a dated citation chain
`test_bench.py`	7	synthetic benchmark: isolated corpus, evidence location, structured `claude -p` output, generator model check
`test_graph.py`	8	corpus graph on a mini-corpus
`test_ligatures.py`	7	repair of lost ligatures
`test_ingest.py`	6	axes, tags and warnings kept on re-extraction
`test_viz.py`	6	figures and detail panel, without a server
`test_status.py`	5	state of the base and next actions
`test_embed.py`	4	embedding cache, wiring into the graph
`test_rag_qwen.py`	4	GPU engine in lexical mode: sources, numerical conditions, stale snapshot, isolated acquisition
`test_reindex.py`	4	benchmark notes, clean measurements

tokenizer is not cached. It is the only failure of the latest run (182 tests out of 183); making it model-independent is still to do.

Two safeguards complement these tests in use: `reindex` validates every file against the schema and stops at the first discrepancy, and a list of manual checks of the ingestion chain (metadata, section order, no leftover headers, fallback without GROBID, cross-checking a note's numbers against the PDF) is kept in the technical documentation.

6.2 The `pdb eval` bench

Unit tests check that the code does what it claims. They do not say whether extraction is good or whether retrieval finds the right passages. `bin/pdb eval` measures this on the real corpus, with no LLM and no network, and without writing anything to the base. It covers four parts: extraction, retrieval, quotation checking and timings. A full run takes a few minutes on CPU. `--save` stores the report as JSON (dated, with the measured commit), and `--compare` reports the differences with the latest report, with an error code if a threshold of `eval/thresholds.json` is no longer met. The procedure is: save before a change (extraction, chunking, model, fusion), make it, then compare. The figures below come from a run of 26/09/2026 on 45 papers and 3 479 passages (`eval/results/2026-09-26T2044-65074b7.json`). Where useful, they are compared with the previous run, made the same day on 28 papers and 2 414 passages: the question sets did not change, only the corpus grew.

6.3 Extraction quality

The bench first computes intrinsic metrics, which need no annotation. It then reuses two ground truths already present in the base: the hand-checked metadata and the values recorded in

`benchmarks.csv` (table 7).

Table 7: Extraction quality (45 papers, all extracted by GROBID; in parentheses, the 28-paper run).

Metric	Value
Text coverage: share of PDF words (bibliography excluded) found in `fulltext.md`	mean 0.91; min. 0.81
Tables whose body was extracted (233 tables, 75 reconstructed by Docling and 4 by PyMuPDF)	0.88 (0.90)
Remaining ligatures, per 10 000 words	0.00
Papers with undetected numbered sections	13 / 45 (7 / 28)
GROBID header alone versus `metadata.json`: title / year / DOI / arXiv	0.91 / 0.42 / 0.62 / 0.72
GROBID header alone: F1 on author names	0.86
`benchmarks.csv` values found in the cited table (588 values)	0.65 (0.74)
same, rows citing tables only (359 values)	0.71 (0.80)
`benchmarks.csv` values found anywhere in the text	0.76 (0.86)

- *Coverage* cannot reach 1: headers, affiliations, text inside vector figures and hyphenated words are deliberately excluded. It is read relatively, from one run or one paper to another.
- *Year and DOI* are low mostly by construction: the arXiv PDF often carries neither the date nor the DOI of the published version. That is why ingestion fills them from arXiv and from citation exports. The bench also revealed a real, still open defect: a publisher template DOI (MDPI) is not rejected.
- *Docling* raised, on 28 papers, the share of tables with a body from 0.66 to 0.90 (41 reconstructed tables, with OCR for three papers whose tables are images); on 45 papers, 75 tables are reconstructed by Docling and the share reaches 0.88. It changed neither the recovered values nor retrieval: the tables cited by the notes already had a body, and the missing values come from cells merged by extraction, prose quoted in the same source section, or a different notation. The quality of OCR'd tables is uneven: a visual check of three tables found one clean, one incomplete and one partly unreadable. Hence the "to be checked" label and the rule of not taking a number from them without rereading the PDF.
- *Recovered values drop* (0.74 → 0.65 in the cited table, 0.86 → 0.76 in the text). The 249 added values come from the 17 new papers, several of which have tables reconstructed by Docling or OCR. The bench does not tell whether the gap comes from the extraction of these tables or from the agent's transcription; both remain to be checked paper by paper, and this part of the bench serves to target them.

6.4 Passage retrieval

Question sets. Each question is linked to the relevant sections of the corpus (identified by `paper#order`, independently of the window chunking). These sections are chosen by text pattern in the target papers (for example *reuse factor* in the two hls4ml papers). Questions rephrase the topic without repeating the pattern when possible: "removes duplicate bounding boxes" for non-maximum suppression. There are three sets:

- *dev*: 20 English questions, used for the design choices (model, fusion, reranking);
- *holdout*: 25 English questions, written afterwards and never used to tune anything. They cover the 12 papers absent from the dev set and include 2 questions on tables and 3 questions spanning several papers. Three expectations were widened after reading the returned

passages, because those passages did answer the question (before that: recall@5 of 0.68 with reranking);

- *FR*: 8 dev questions translated into French by hand.

Metrics. recall@5 and recall@10 (at least one expected section among the top 5 or 10 passages), MRR (inverse of the rank of the first correct section), nDCG@10 with binary gain (Järvelin and Kekäläinen, 2002), which rewards finding *several* relevant sections, and paper@5 (at least one passage from a target paper among the top 5).

Table 8: Passage retrieval over 3 479 passages (45 papers, bge-small-en-v1.5 model), 26/09/2026. Bold: best value per set.

Set	Mode	recall@5	recall@10	MRR	nDCG@10	paper@5
dev (20)	lexical (BM25)	0.95	0.95	0.75	0.37	**1.00**
	dense	0.90	0.90	0.70	0.36	0.95
	hybrid (default)	0.90	0.95	0.76	0.39	0.95
	hybrid + reranking	**1.00**	**1.00**	**0.85**	**0.44**	**1.00**
holdout (25)	lexical (BM25)	0.60	0.76	0.57	0.27	0.68
	dense	0.64	0.72	0.56	0.32	0.76
	hybrid (default)	0.72	**0.80**	0.60	0.31	0.80
	hybrid + reranking	**0.80**	**0.80**	**0.71**	**0.38**	**0.88**
FR (8)	lexical (BM25)	0.12	0.38	0.06	0.04	0.25
	dense	**0.62**	**0.75**	**0.38**	**0.17**	**0.75**
	hybrid (default)	0.50	**0.75**	0.37	0.15	**0.75**
	hybrid + reranking	0.50	0.50	0.32	0.12	0.62

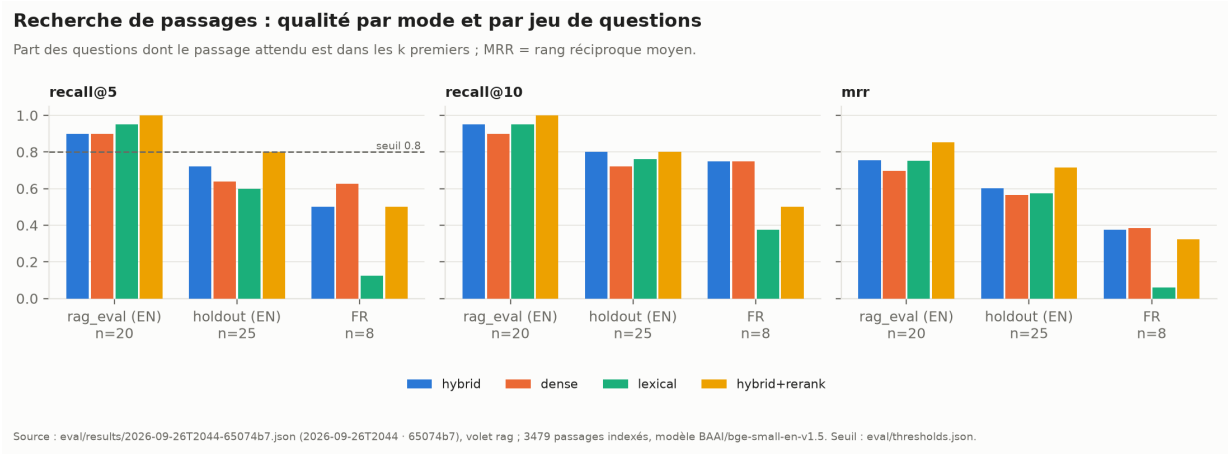


Figure 3: The numbers of table 8, plotted by `pdb plot eval` from the bench's JSON report (labels in French). The dashed line is the acceptance threshold (recall@5 $\geq$ 0.8).

Reading the results.

- *The dev set was optimistic.* On the holdout, recall@5 loses 18 to 35 points depending on the mode. Lexical goes from first to last (0.95 $\rightarrow$ 0.60). This confirms the expected bias: on the dev set, the expected sections had been chosen by keywords, which favoured BM25.
- *More papers, more competitors.* With identical questions, going from 28 to 45 papers barely changes the dev set, but lowers the holdout in the three modes without reranking: hybrid

0.80 → 0.72, dense 0.76 → 0.64, lexical 0.68 → 0.60. The new papers bring passages close to the topic that outrank the expected passages. Reranking absorbs this competition at recall@5 (0.80 in both runs) but not entirely in rank (MRR 0.78 → 0.71). Expectations were not revised: a passage from a new paper that answers equally well counts as a failure.

- *Reranking becomes necessary in English.* On 28 papers, hybrid alone reached the threshold on the holdout; on 45, only hybrid with reranking does (0.80). On the dev set, it recovers the two missed questions, including the one on removing duplicate boxes, which contains neither "NMS" nor "non-maximum suppression".
- *Real holdout failures* (with reranking): radiation tolerance, quantisation-aware training, dataset distillation, and the two table questions (autoencoder resources in an hls4ml paper, FINN prototypes). The text of a table is short, often reduced to its caption, and is hard to retrieve.
- *In French, lexical collapses* (recall@5 of 0.12), and the dense index holds up better (0.62), although the model is not designed as multilingual. Reranking, trained on English MS MARCO, degrades the ranking. The GPU engine (section 4.6) does much better on this set.
- *Decisions.* Hybrid remains the default mode of `pdb search`, because reranking degrades French, following a rule set before the measurement. `/paper-ask` enables it, because it always phrases its queries in English and the rank of the first correct answer matters when only 2 or 3 passages are opened. The new run reinforces this choice.

In an earlier measurement (25/09/2026, dev set), the `all-MiniLM-L6-v2` model reached a recall@5 of 0.75 in dense mode versus 0.90 for bge-small, which motivated choosing the latter. The acceptance threshold is a recall@5 of at least 0.8 in English: on 45 papers, the holdout only reaches it with reranking.

6.5 Detecting that a question falls outside the corpus

Can one tell, without an LLM, that the base does not cover a question? Two sets of 10 out-of-corpus questions (protein folding, blockchain, etc.) were written: one for calibration, the other for validation. The second was written before any cross-encoder measurement. Two signals are measured: the best cosine of dense retrieval, and the best cross-encoder score over the top 3 passages. For each, the threshold is chosen on the calibration data (dev set and first out-of-corpus set, from which a question half-covered by the corpus was removed) by maximising balanced accuracy, then applied as is to the validation data (holdout and second out-of-corpus set). The adoption rule was set before the measurement: on validation, at least 80 % of out-of-corpus questions flagged, and at most 5 % of corpus questions.

Table 9: "Out-of-corpus" signal: share of questions flagged on validation (10 out-of-corpus questions, 25 holdout questions).

Signal	Threshold	Out-of-corpus flagged	Corpus wrongly flagged	Adopted
cosine, initial hand-set threshold	0.60	0.00	0.00	–
cosine, calibrated threshold	0.78	0.80	0.20	no
cross-encoder, calibrated threshold	2.41	0.90	0.40	no
Qwen3-Reranker-4B, 28 papers (section 4.6)	0.96	1.00	0.16	no

No signal meets the rule. With bge-small, an off-topic question gets a cosine as high as a relevant one: the initial calibration, made on a few queries, did not hold, and the warning never fired. The cross-encoder separates the medians better, but some corpus questions score very low, often those where retrieval already fails. The warning was therefore removed from `pdb`

`search`. The bench keeps measuring both signals, for a future attempt to be validated on a new out-of-corpus set. The GPU engine's reranker is the signal closest to the rule: it flags all 10 out-of-corpus questions, but also 4 of the 25 holdout questions (16 %, for 5 % allowed). It is not adopted either.

6.6 Quotation checking

The bench draws 40 random passages (fixed seed) and builds four kinds of quotations: an exact sentence, which must be accepted; the same sentence with one word replaced, then with one number changed, and a sentence from another paper, which must be rejected. "Accepted" means a zero exit code (`OK` or `APPROX` in the cited passage).

Table 10: Acceptance rate of `pdb verify`, before and after the number check. "Before": 28-paper run; "After": 45-paper run, whose random draw differs (35 sentences with a changeable number instead of 38).

Quotation	Before	After	Statuses after
exact (40)	1.00	1.00	40 `OK`
number changed (35)	**1.00**	**0.00**	35 `NOMBRES`
word replaced (40)	0.97	0.88	35 `APPROX`, 3 `NOMBRES`, 2 `ABSENT`
from another paper (40)	0.00	0.00	40 `ABSENT`

Before the check, `verify` accepted every quotation with a changed number, whereas the agent's working rules rely on it to guarantee quoted numbers. A first version of the check still let 6 through (a "4" recognised in "6.4", a "2" found elsewhere in the passage). Requiring neighbouring words fixes both cases. Still not handled: a *word* replaced far from any number still yields `APPROX`. That is why `/paper-ask` keeps an `APPROX` quotation only when presented as a paraphrase.

6.7 Timings

Table 11: Measured timings (medians of 3 repetitions, CPU without GPU).

Measurement	Value
Extraction outside GROBID, per paper (TEI → text, ligatures, tables)	0.01 to 0.13 s
Index loading (3 479 passages, 7 MB)	0.21 s
bge-small encoding	16 passages/s; full index ≈ 223 s
Warm `search`, p50 / p95: lexical	8 / 16 ms
dense	116 / 143 ms
hybrid	113 / 128 ms
hybrid + reranking	1 242 / 1 470 ms
Warm `verify`	274 ms
Cold commands, Python start-up included: `status`	0.12 s
`bench`, `paper`	0.05 s
`show`	0.36 s
`search`	5.3 s

The cost of a search comes almost entirely from start-up (PyTorch import, model loading): 5.3 s cold versus 0.1 s warm. This is the real cost of a call by the agent, which launches a command each time. With 44 % more passages, warm timings stay of the same order, except `verify` (4 → 274 ms). This gap has not been analysed yet; it remains small compared with the cost of a cold command.

6.8 Real end-to-end trial

Question asked to `/paper-ask`: "how do the papers measure power, and is it comparable?". The agent chained one `bench` query (57 rows, 8 papers), one search of 10 passages, reading 3 of them, 7 quotation checks and 8 identity queries. The whole amounts to about 9 kB read, i.e. about 2 500 tokens, without opening any full text. All 7 verbatim quotations came back `OK` at the first attempt. Retrieval found in one query the four measurement protocols of the corpus (PMBus and wall socket, multimeter at the board input, `tegrastats`, idle power measured before the tests). The Read / Inferred / Not found structure brought out two points to check in the base itself, including a paper that describes a careful measurement while no power row had been entered.

The trial also revealed two defects, fixed since and each covered by a test: a short identifier truncated for a compound author name, and an aggregation that required an external script (hence `bench --count`).

6.9 Synthetic benchmark (`pdb synth`)

The previous sets are small and written by the author of the system. `bin/pdb synth` prepares a larger evaluation, kept apart from the working base (`.cache/bench/`):

- a *frozen corpus* of 200 papers, with neighbouring topics as distractors; question anchor passages are screened for FPGA, embedded AI and hardware acceleration, excluding introductions and background;
- *local generation* of questions by an open model (Qwen3.8-27B quantised to Q6, served by llama.cpp on the same RTX 5090), as schema-constrained JSON with a fixed seed; the identity of the served model and every quotation are checked;
- *validation without human labels*: two independent sessions of an LLM (Codex for the 200 reference questions, then Claude Code in non-interactive mode, `claude -p`) check each question;
- *quotas*: 40 single-passage, 40 numerical, 50 cross-paper, 30 within-paper, 20 table and 20 missing-evidence questions; a 40 % development, 60 % test split over disjoint paper sets, balanced between English and French, and 40 questions declined into translated and terse variants;
- *ablations*: the four retrieval modes of the GPU engine and the two CPU modes of `pdb search` on the same questions, then agent variants run in a sandbox (bubblewrap, read-only tools, read-only corpus), scored by an LLM judge.

Automatically produced notes and numerical transcriptions form a separate *enriched* corpus, never used to write reference answers; a benchmark row is kept there only if each of its numbers appears in the cited section. The protocol and its tests are in place; its results are not available yet and are therefore not reported here.

6.10 Threats to validity

- The sets are small: on 25 questions, a difference of 0.04 is a single question. The holdout tuned no parameter, but it was written, like the dev set, by the author of the system. Questions written by a third party are needed to confirm the differences.
- Expected sections are chosen by text pattern. This is an approximation: a correct answer phrased differently counts as a failure (three holdout expectations had to be widened for this reason).
- The ground truth for tables only covers values taken into `benchmarks.csv`.

- The quality of the answers written by the agent is not evaluated: the end-to-end trial is a single one, judged by the agent that ran it. An independent evaluation would require human judgement.
- The comparison of the two engines is on the 28-paper corpus; it was not redone after adding 17 papers, which lowered the CPU engine.
- The synthetic benchmark will have its own biases: questions generated by an LLM, validated and scored by other LLMs. It will measure this frozen corpus and this task distribution, not research work in general.

7 Limitations and outlook

- **Tables.** They remain the most fragile part of extraction. Docling brings the share of tables with a body to about 90 %, but OCR'd tables are of uneven quality and stay labelled "to be checked". Tables are hard to retrieve, and a number missing from `benchmarks.csv` and held in a badly extracted table will not be found reliably: `bench` remains the reference for numbers.
- **Out-of-corpus questions.** `search` always returns passages, and no tested score reliably tells that the base does not cover the topic. That judgement is left to the agent. The Qwen reranker comes close to the rule (16 % false alarms for 5 % allowed). Leads: combine cosine and cross-encoder, then validate on a new out-of-corpus set.
- **Verification.** `verify` guarantees numbers, but a word replaced far from any number still passes as a paraphrase (`APPROX`).
- **Metadata.** A publisher template DOI (MDPI) is not yet rejected at extraction; it is fixed by hand in the base, but a new paper from the same publisher would inherit it.
- **Language.** The corpus and the default model are in English, and CPU retrieval loses a lot in French. The GPU engine (Qwen3) retrieves all 8 French questions, but at the price of a 32 GB GPU. Lead: a small multilingual model on CPU, compared with the same evaluation.
- **Scaling.** From 28 to 45 papers, holdout recall@5 in hybrid mode goes from 0.80 to 0.72; only reranking holds the threshold. The trend must be monitored as the corpus grows, with `pdb eval --compare`, and the expectations of the question sets will need revising when a new paper answers equally well.
- **Tests.** One acquisition test still depends on the cached Qwen tokenizer.
- **Latency.** About 5.3 s per cold `search`, versus 0.1 s warm, mostly due to the PyTorch import and model loading; `show` and `verify`, which do without them, answer in about 0.3 s. A resident process, for example an MCP server exposing the same commands, would pay this cost only once and would let other applications use the base. The same argument applies to the GPU engine, which reloads its models for each query (3 to 4 s).
- **Genericity.** Nothing in the retrieval layer depends on the FPGA domain; only the `bench` filter shortcuts, the benchmark columns, the axes and the evaluation set are specific to the corpus. Adapting the tool to another topic amounts to redefining these constants and the agent's working rules.

8 Conclusion

PaperDatabase shows that a modest research corpus can be made usable by an LLM agent without a database or an external service: well-structured text files, a table of sourced numbers and a deterministic hybrid search are enough. Generation stays with the agent, but everything it quotes can be replayed and checked by a command. The cost of a sourced answer drops by an order of magnitude compared with reading full texts. The evaluation bench also showed the value of measuring what one believes settled: it revealed an optimistic question set, a warning that never fired and a check that let wrong numbers through. The last two defects led to removing the warning and fixing the check. Remeasuring after adding 17 papers, it also showed that retrieval without reranking loses recall as the corpus grows. A second GPU engine solves most of the French problem, without replacing the first. Next steps are the results of the synthetic benchmark, questions written by a third party, better retrieval within tables, out-of-corpus detection and a resident engine that avoids reloading models for each query.

References

Christoph Auer, Maksym Lysak, Ahmed Nassar, Michele Dolfi, Nikolaos Livathinos, Panos Vagenas, Cesar Berrospi Ramis, Matteo Omenetti, et al. Docling technical report. arXiv:2408.09869, 2024. Preprint.

Gordon V. Cormack, Charles L. A. Clarke, and Stefan Büttcher. Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 758–759, 2009. doi: 10.1145/1571941.1572114.

GROBID contributors. GROBID: machine learning software for extracting information from scholarly documents. `https://github.com/kermitt2/grobid`, 2008–2026. Logiciel, version 0.8.1 utilisée.

Kalervo Järvelin and Jaana Kekäläinen. Cumulated gain-based evaluation of IR techniques. *ACM Transactions on Information Systems*, 20(4):422–446, 2002. doi: 10.1145/582415.582418.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, 2020.

Tri Nguyen, Mir Rosenberg, Xia Song, Jianfeng Gao, Saurabh Tiwary, Rangan Majumder, and Li Deng. MS MARCO: A human generated MAchine reading COmprehension dataset. In *Workshop on Cognitive Computation (CoCo@NIPS)*, 2016.

Rodrigo Nogueira and Kyunghyun Cho. Passage re-ranking with BERT. arXiv:1901.04085, 2019. Preprint.

Jason Priem, Heather Piwowar, and Richard Orr. OpenAlex: A fully-open index of scholarly works, authors, venues, institutions, and concepts. arXiv:2205.01833, 2022. Preprint.

Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, 2019. doi: 10.18653/v1/D19-1410.

Stephen E. Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009. doi: 10.1561/1500000019.

Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, 2020.

Shitao Xiao, Zheng Liu, Peitian Zhang, Niklas Muennighoff, Defu Lian, and Jian-Yun Nie. C-Pack: Packed resources for general Chinese embeddings. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2024. doi: 10.1145/3626772.3657878. Preprint arXiv:2309.07597 (2023).

A Acronyms

Acronym	Meaning
AI	artificial intelligence
API	Application Programming Interface
BF16	bfloat16, 16-bit floating-point format
BM25	Best Matching 25, scoring function of lexical retrieval
BRAM	Block RAM, on-chip memory of an FPGA
CPU	Central Processing Unit
CSV / TSV	Comma- / Tab-Separated Values
DOI	Digital Object Identifier, persistent identifier of a publication
DSP	Digital Signal Processing: arithmetic block (multiply-accumulate) of an FPGA
FPGA	Field-Programmable Gate Array
FPS	Frames Per Second
FTS5	Full-Text Search version 5, SQLite full-text index
GOPS	Giga Operations Per Second
GPU	Graphics Processing Unit
GROBID	GeneRation Of BIbliographic Data, structure extractor for PDF papers
INT8	8-bit integers (quantisation)
JSON	JavaScript Object Notation
LLM	Large Language Model
LUT	Look-Up Table, elementary logic cell of an FPGA
MCP	Model Context Protocol, protocol exposing tools to an LLM agent
MRR	Mean Reciprocal Rank of the first correct answer
MS MARCO	Microsoft MAchine Reading COmprehension, passage-retrieval dataset
nDCG	normalized Discounted Cumulative Gain, ranking quality
NMS	Non-Maximum Suppression, removal of duplicate detection boxes
OCR	Optical Character Recognition
PDF	Portable Document Format
RAG	Retrieval-Augmented Generation
RRF	Reciprocal Rank Fusion
SHA-1	Secure Hash Algorithm 1, hash of a text
TEI	Text Encoding Initiative, XML output format of GROBID
YOLO	You Only Look Once, family of object detectors

B Glossary

Term	Meaning in this document
agent	LLM that calls tools (file reading, commands) to carry out a task; here Claude Code
benchmark row	one configuration reported by a paper (model, platform, precision, throughput, power…), with its source section, in `benchmarks.csv`
chunk, passage	section fragment indexed for retrieval, identified by `<paper>#<section>.<k>`
cross-encoder	model that scores a query and a passage together; used for reranking
embedding	vector representing the meaning of a text; close texts have close vectors
gap	area the corpus does not cover, recorded in `gaps.md`
holdout	reserved question set, never used to tune any parameter
dense / lexical index	index by embeddings (meaning) / by exact words (BM25, FTS5)
`n/r`	"not reported": value absent from the paper, never estimated
paper@k	share of questions for which an expected paper is among the top k passages
power scope	what a power measurement covers (chip, board, estimate), `power_scope` column
preprint	paper released without peer review (arXiv)
reading note (`fiche`)	critical reading of a paper (`fiche.md`): problem, method, results, limitations, relevance
recall@k	share of questions for which an expected section is among the top k passages
reranking	reordering of the top results by a more expensive model
skill	written procedure the agent follows for a task (`/paper-ask`, `/paper-fiche`…)
toolflow	tool chain turning a neural network into a hardware accelerator (FINN, hls4ml…)

C Commands

Command	Effect
`bin/pdb status`	state of the base and next actions
`bin/pdb add <arxiv | doi | pdf>`	acquisition, ingestion and reindexing
`bin/pdb ingest [--force | --reparse]`	(re)extraction of the PDFs in `inbox/`
`bin/pdb ingest --reparse --tables docling [--ocr]`	bodiless tables handed to Docling
`bin/pdb reindex`	`INDEX.md`, `external_refs.csv`, passage index
`bin/pdb bench [filters]`	filters `benchmarks.csv`, source columns always shown
`bin/pdb paper <id | word>...`	identity of one or more papers
`bin/pdb search "<q>"`	ranked passages (hybrid, optional `--rerank`)
`bin/pdb show <chunk_id>`	exact text of a passage
`bin/pdb verify <chunk_id> "<quote>"`	checks a quotation
`bin/pdb cites | cited-by <id>`	internal citation graph
`bin/pdb discover`	candidates to add, via OpenAlex
`bin/pdb plot [bench | eval | corpus | all]`	static figures and `figures/manifest.json`
`bin/pdb viz`	local corpus visualisation
`bin/pdb rag index | search | evidence | benchmarks`	GPU engine (Qwen3): index, search, evidence, numbers
`bin/pdb rag acquire | approve | check-answer`	open-access acquisition, import after approval, answer check
`bin/pdb synth <step>`	synthetic benchmark (corpus, generation, validation, ablations, report)
`bin/pdb test`	unit tests
`bin/pdb eval [--extract | --rag | --speed]`	evaluation bench, by part or complete
`bin/pdb eval --save | --compare`	saves a report; compares with the latest one and the thresholds
`/paper-add`, `/paper-fiche`	agent skills: addition; reading note and benchmark rows
`/paper-ask`, `/paper-gaps`, `/paper-discover`	sourced answer; gap map; discovery
`/paper-research`	sourced answer with the GPU engine and open-access acquisition

Installation. Python 3.12 and Docker (for GROBID):

```
bash bin/setup.sh                                  # .venv and dependencies
.venv/bin/pip install -r requirements-embed.txt    # optional: dense retrieval
bin/pdb grobid start                               # GROBID on localhost:8070
bin/pdb add 2507.18174 && bin/pdb test
```